

Microcontroladores

Roteiro 0 — Bancada

Raoni F. S. Teixeira · DENE/UFMT · 1 sessão · 1,0 ponto · grupos de 3

Objetivos desta sessão

Ao final desta sessão o grupo deve ser capaz de:

1. Identificar as seções do kit XM118 e localizar o microcontrolador;
2. Criar e compilar um projeto em C no MPLAB X com o compilador XC8;
3. Gravar o firmware no PIC18F4550 pelo bootloader USB;
4. Explicar por que os atuadores precisam de estado de repouso seguro;
5. Prever e medir no osciloscópio a frequência da onda gerada em RD0.

Parte 1 — Reconhecimento do kit

Tarefa

Localize e anote a posição de cada item:

1. O microcontrolador (CI de 40 pinos, seção MICROCONTROLLER);
2. Os bancos de chaves DIP CH1 a CH5 e a chave U8;
3. A seção PUSH BUTTONS e, dentro dela, o botão RESET (SW9);
4. Os 8 LEDs da seção LEDS;
5. O display LCD alfanumérico;
6. A ventoinha e a resistência de aquecimento;
7. O conector USB frontal e o DB9 da seção RS232.

Ligue o kit **sem nada conectado** e observe:

1.1 Os LEDs indicadores de alimentação acendem?

1.2 O LCD acende?

Parte 2 — Compilar

Criar o projeto

1. MPLAB X → **File** → **New Project** (Ctrl+Shift+N).
2. Categoria **Microchip Embedded**, tipo **Application Project(s)**. Avançar.
3. Em **Device**, digite PIC18F4550. Confira que a família selecionada é **Advanced 8-bit MCUs (PIC18)**. Avançar.
4. Em **Select Tool**, escolha **No Tool**. Avançar.
5. Em **Select Compiler**, escolha o **XC8** disponível na máquina. Avançar.
6. Em **Project Name**, use r0_blink. Confira a pasta de destino e conclua em **Finish**.

Observação

Em versões mais antigas do MPLAB X, o tipo de projeto aparece como **Standalone Project** em vez de **Application Project(s)**. É o mesmo caminho.

Se o MPLAB X pedir para baixar o *Device Family Pack* do PIC18F4550, autorize. Sem ele o compilador não encontra o cabeçalho do dispositivo.

Atenção

O caminho da pasta do projeto não pode conter **acentos, espaços ou cedilha**. Um caminho como C:\Users\Joao Carlos\Meus Projetos\ faz o XC8 falhar com mensagens que não têm relação aparente com a causa. Use algo como C:\micro\r0_blink.

Adicionar o código

Botão direito em **Source Files** → **Add Existing Item**, e adicione o arquivo r0_blink.c fornecido pelo professor.

O trecho essencial do programa é este:

```
#include <xc.h>
#define _XTAL_FREQ 16000000UL

void main(void) {
    LATD = 0xFF; /* valor primeiro: os 8 LEDs nascem acesos */
    TRISD = 0x00; /* direção depois: PORTD vira saída */

    while (1) {
        LATDbits.LATD0 = 1;
        __delay_ms(500);
        LATDbits.LATD0 = 0;
        __delay_ms(500);
    }
}
```

Sete LEDs permanecem acesos; apenas o LED ligado a **RD0** pisca. Esse pino é o sinal que você vai medir na Parte 5.

Divergência do manual

Diretivas `#pragma` config escritas na aplicação são **silenciosamente ignoradas** neste kit: os bits de configuração pertencem ao bootloader, que já os fixou. O bootloader configura 48 MHz para o periférico USB e **16 MHz para a CPU**. Por isso `_XTAL_FREQ` vale 16000000UL, e não 48000000UL.

Compilar

Use o **martelo** da barra de ferramentas (*Clean and Build*, F11).

Atenção

Não use o botão de Debug. O build de depuração gera `.elf` mas **não gera** `.hex`, e sem `.hex` não há o que gravar.

Este é o erro mais frequente da sessão.

O arquivo deve estar em:

```
<projeto>/dist/default/production/<nome>.production.hex
```

Se existir apenas a pasta debug, você compilou no modo errado. Volte e use o martelo.

2.1 Anote o caminho completo do arquivo gerado.

Parte 3 — Gravar

1. Conecte o cabo USB na **porta frontal do kit (CN9)**.
2. Abra o programa **Bootloader PIC18 — XM118** (USB HID Bootloader).
3. Pressione o **botão de reset (SW9)** do kit.
4. Clique em **Connect** dentro de 4 a 5 segundos.
5. **Buscar arq HEX** → selecione o `.production.hex` da Parte 2.
6. **Begin uploading**.
7. Ao final, pressione **RESET (SW9)** para executar o firmware novo.

A janela de histórico deve mostrar a sequência:

```
Flash Erase...  
Flash Write...  
Completed successfully.  
Disconnected.  
Reset device to reenter bootloader mode.
```

Observação

Repare na última mensagem do próprio programa: *reset device to reenter bootloader mode*. Ela confirma o que foi explicado na primeira seção — para gravar de novo, é preciso reiniciar de novo.

Se não conectar

Na ordem:

1. **Reinicie o kit e clique em Connect dentro de 4 a 5 segundos.** A causa mais frequente é a janela do bootloader ter expirado. O programa não avisa: apenas não encontra o dispositivo.
2. Confirme que o cabo está na USB **frontal**, não na traseira.
3. Feche o MPLAB X antes de tentar.
4. Desconecte e reconecte o cabo com o programa fechado.

3.1 Quantas tentativas o grupo precisou até conectar na primeira vez? O que estava errado?

Parte 4 — Estado de repouso seguro

Com o *blink* rodando, responda:

4.1 Sete LEDs ficam acesos e um pisca. Qual deles pisca, e com que período aproximado?

4.2 Localize no código as duas linhas abaixo e explique, com suas palavras, o que cada uma faz.

```
LATC  &= ~0b00000110;  /* linha A */
TRISC &= ~0b00000110;  /* linha B */
```

Previsão — registre ANTES de medir

4.4 — O que aconteceria se as linhas A e B fossem trocadas de ordem?

Pense no que o pino faz no instante em que deixa de ser entrada e passa a ser saída. Formule a resposta **antes** de testar; depois, se quiser, teste.

Conceito central

A pergunta 4.4 é o conceito central deste roteiro. Com uma ventoinha, o efeito de um pulso indesejado é inofensivo. Com a resistência de aquecimento — ou, em um sistema de potência real, com um disjuntor ou uma chave de manobra — não seria.

Escrever no registrador de dados (LAT) **antes** de configurar a direção (TRIS) é a diferença entre um atuador que acorda desligado e um que acorda em estado indefinido.

Parte 5 — Medindo o pisca no osciloscópio

Até aqui você julgou o programa pelo que o olho vê. Agora vai medir o mesmo sinal com um instrumento, e comparar a medida com um número calculado antes.

Onde medir

O pino que pisca é o **RD0**, na seção **LEDS** do kit. Meça o sinal lógico nesse ponto, com a garra de terra em um ponto de **GND** da placa.

Perigo — leia antes de encostar a ponta de prova

Os pontos de teste **LAMP**, **HEATER** e **COOLER** estão no **trilho de 12 V**. A garra de terra do osciloscópio **nunca** pode encostar neles: a garra está ligada ao terra da rede pelo cabo de força, e o contato cria um curto através do instrumento.

Nesta parte você mede **apenas RD0**, que é um pino lógico de 5 V. Se a ponta de prova precisar ir a qualquer outro lugar, chame o professor antes.

Configuração de bancada

Ponta de prova em **1×** ou **10×** (anote qual). Acoplamento **DC**. Base de tempo inicial em torno de **200 ms/div** para enxergar o período de 1 s.

De onde vem o número previsto

O compilador precisa de `_XTAL_FREQ` para converter um atraso em milissegundos num número de ciclos de máquina. Com $F_{osc} = 16 \text{ MHz}$:

$$T_{cy} = \frac{4}{F_{osc}} = \frac{4}{16 \times 10^6} = 250 \text{ ns}$$

O laço acende por 500 ms e apaga por 500 ms. O período da onda é a soma das duas metades, e a frequência é o seu inverso.

Previsão — registre ANTES de medir

5.1 — Preencha as colunas de período e frequência previstos para os quatro valores da tabela, **antes de ligar a ponta de prova**. Rubrique e mostre ao professor.

A nota é pelo raciocínio registrado, não pelo acerto.

<code>__delay_ms</code> (cada metade)	Período previsto	f prevista	f medida	Erro (%)
500				
250				
100				
50				

Tabela 1: Previsão e medição da onda quadrada em RD0.

Tarefa

5.2 — Meça o período com os cursores e preencha as duas últimas colunas. Cada linha exige alterar o valor no código, recompilar (martelo) e regravar (reset, Connect, upload, reset).

5.3 — O erro percentual é aproximadamente o mesmo nas quatro linhas ou cresce com a frequência? O que isso diz sobre a origem do erro: uma fonte proporcional ao período (o clock) ou uma fonte de valor fixo (as instruções do laço que não são o atraso)?

Descobrimo o clock pela medida

Suponha que um colega tenha declarado `_XTAL_FREQ` como `48000000UL`, por ter lido no manual que o kit opera a 48 MHz. O compilador acredita na declaração e calcula os ciclos do atraso a partir dela.

Previsão — registre ANTES de medir

5.4 — Se o clock real da CPU é 16 MHz e o compilador supôs 48 MHz, o atraso executado será mais **curto** ou mais **longo** que o pedido, e por qual fator? Deduza antes de testar.

Tarefa

5.5 — Volte o atraso para 500 ms, troque `_XTAL_FREQ` para `48000000UL`, recompile, grave e meça o novo período. A razão entre o período medido e o previsto confirma sua dedução?

5.6 — Inverta o raciocínio: partindo apenas de um período medido no osciloscópio e do valor declarado em `_XTAL_FREQ`, escreva a expressão que recupera a frequência real da CPU. Este é o método que você usará sempre que suspeitar do clock.

Atenção

Ao terminar, **devolva** `_XTAL_FREQ` para `16000000UL` e regrave. Sair da sessão com o valor errado quebra todos os roteiros seguintes.

Observação

Toda temporização do semestre — o Timer0 do termostato, o *baud rate* da comunicação serial, a frequência do PWM — depende de `_XTAL_FREQ` estar correto. Um clock supostamente errado é a primeira hipótese a testar quando um periférico temporizado se comporta de forma estranha.

O limite do olho**Tarefa**

5.7 — Reduza o atraso progressivamente e registre a partir de qual valor o LED **parece continuamente aceso**, sem piscar perceptível. Converta esse atraso em frequência.

5.8 — Nessa condição, o LED ainda está piscando? O que o osciloscópio mostra? Explique por que, acima dessa frequência, o osciloscópio deixa de ser um instrumento conveniente e passa a ser o **único** capaz de revelar o que o pino está fazendo.

Conceito central

Você acaba de encontrar experimentalmente o princípio do controle por largura de pulso: acima de algumas dezenas de hertz, a carga — o olho, um motor, uma resistência de aquecimento — responde à **média** do sinal, não a cada transição. Esse mesmo fenômeno reaparece mais adiante no curso, com o hardware do módulo CCP gerando a onda no lugar do laço de atraso.

Teste de aceitação

O roteiro está concluído quando, na presença do professor:

- ☐ Sete LEDs do PORTD permanecem acesos e o LED de RD0 pisca.
- ☐ O grupo mostra o arquivo `.production.hex` que gerou.
- ☐ O grupo grava novamente o firmware, do zero, sem consultar o roteiro.
- ☐ A tabela de previsão está preenchida, com as medidas ao lado.
- ☐ `_XTAL_FREQ` foi devolvido a 16000000UL e o kit regravado.

Entrega e critério

Peso	Item
0,20	Teste de aceitação aprovado em bancada
0,10	Resposta 3.1 — tentativas até conectar e diagnóstico
0,15	Respostas 4.1 a 4.3
0,25	Resposta 4.4, com justificativa do mecanismo
0,30	Parte 5 — previsão rubricada, medidas e respostas 5.3 a 5.6

Tabela 2: Distribuição do ponto do Roteiro 0.

As respostas 4.4 e 5.6 valem mais que as demais porque são as únicas que exigem raciocínio sobre causa, e não observação.

Armadilhas frequentes

Sintoma	Causa provável
Não existe pasta <code>production</code>	Compilou pelo botão de Debug
<code>PK2Error0022</code>	Tentou gravar pelo PICKit-2; use o bootloader
Bootloader não conecta	Janela de 4 a 5 s expirou: reinicie e clique em seguida
Bootloader não conecta (persistente)	MPLAB X aberto, ou cabo na USB traseira
LED não pisca após gravar	Faltou o RESET (SW9)
Ventoinha continua ligada	O <code>.hex</code> gravado não é o que você compilou
XC8 falha com erro sem sentido	Caminho do projeto com acento ou espaço
Período medido é 3× menor que o previsto	<code>_XTAL_FREQ</code> declarado como 48 MHz
Osciloscópio mostra linha reta	Garra de terra fora do GND, ou ponta no pino errado

Tabela 3: Diagnóstico rápido do Roteiro 0.

Para a próxima sessão

Tarefa

Traga escrito: qual a diferença entre $LATD = 0x01$ e $LATD \neq 0x01$?

Se não souber, escreva o que você **acha** que acontece em cada caso. A hipótese errada é material de aula — e vale nota tanto quanto a certa.